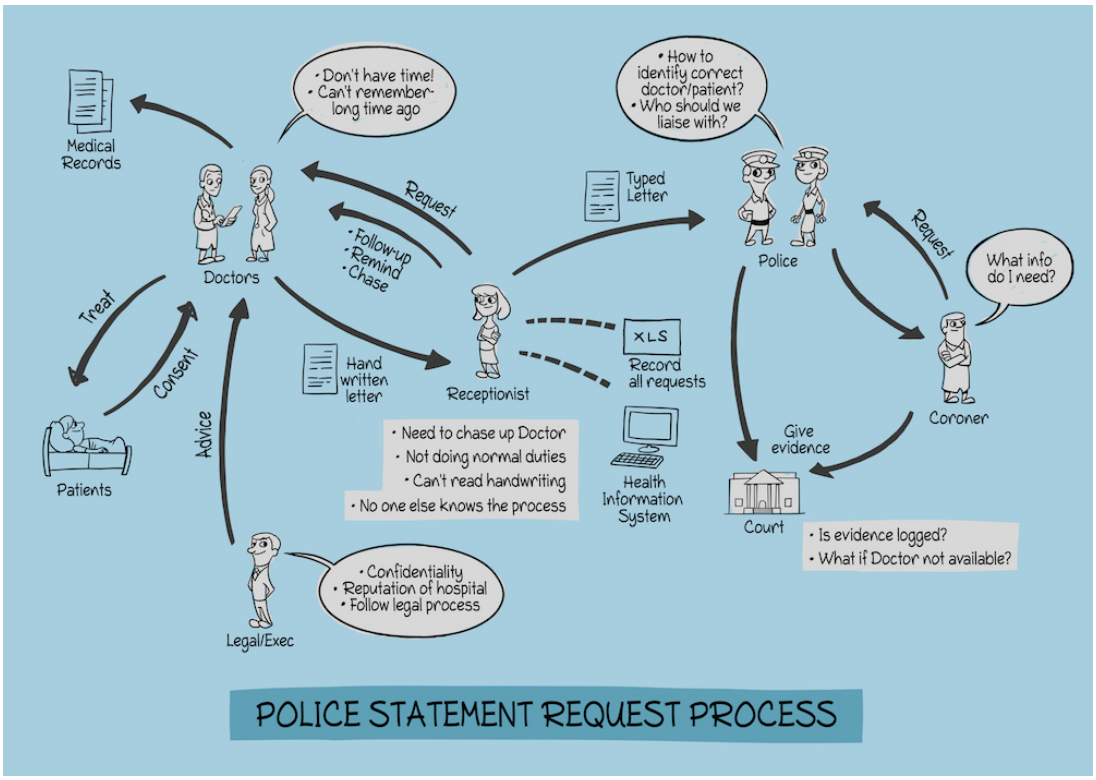
AARHUS UNIVERSITET
Institut for Elektro- og Computerteknologi

Min rapporttitel

Fag/Projekt - Gruppenummer



Forfattere
Dit navn - studienummer
Dit navn - studienummer

Vejleder
Hendes Navn

Antal anslag: 124
19. AUGUST 2026

Abstract

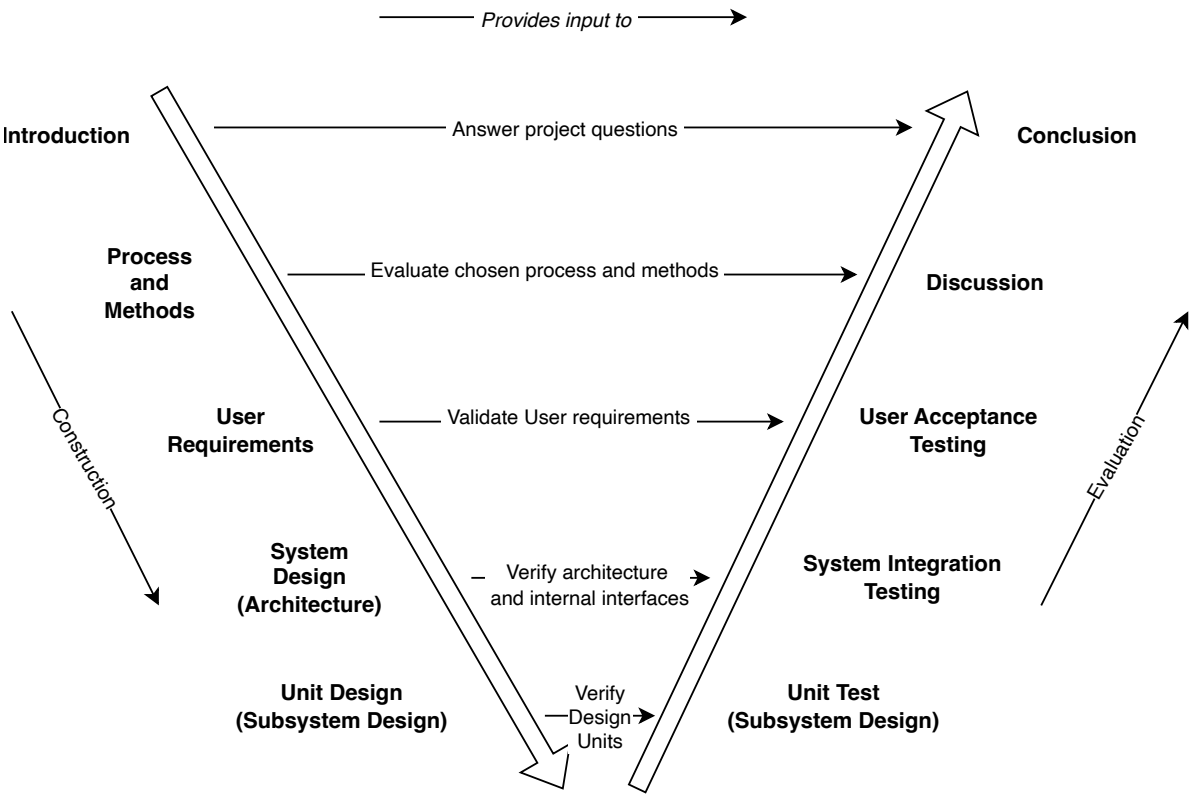
An academic abstract typically outlines four elements relevant to the completed work:

- The research focus (statement of the problem(s)/specific gap in existing research/research issue(s) addressed);
- The research methods (experimental research, case studies, questionnaires, etc) used to solve the problem;
- The major results/findings of the research;[20] and
- The main conclusions and recommendations (i.e., how the work answers the proposed research problem).

Wikipedia

Læsevejledning

Rapportens struktur er variant af IMRAD tilpasset ingeniørprojekter. Strukturen benytter i V-modellen, som vist i Figur 1. I forbindelse med rapporten er modellen udvidet med de akademiske dele, nemlig: *Indledning*, *Metode*, *Diskussion* og *Konklusion*.



Figur 1: Rapportstruktur baseret på V-Modellen

I grove træk beskriver rapporten projektets forløb, fra idé, over krav, til arkitektur, design, test og evaluering.

Når du skriver rapporten så tænk det som at du skriver hvert afsnit, som stod du på netop det sted i projektet. I *indledning* og *metode* afsnittene beskriver du hvad du agter at gøre og hvordan. I *krav* beskriver du hvilke krav du vil opfylde. I *arkitektur* beskriver du hvordan du vil lave din arkitektur, baseret på kravene. I *design* beskriver du hvordan du designer og implementerer løsningens komponenter. I testafsnittene tester du og kigger tilbage på resultaterne. Diskussionsafsnittet kigger tilbage på projekter og diskuterer konsekvenserne af de valg i tog undervejs og til sidst konkluderer du på projektet, hvordan det gik i forhold til de mål som du satte op indledningsvist. Pas derfor på med i metodeafsnittet at skrive "vi brugte Scrum...", da det skuer tilbage. I stedet skal intentionen beskrives. Hvordan det gik med Scrum kan diskuteres til sidst, hvor der netop skues tilbage.

Det bliver en lidt plandrevet beskrivelse af projektet, også selvom man har arbejdet ved hjælp af en iterativ metode, men det giver en naturlig opbygning af argumenter og evaluering af resultater, som man kender fra IMRAD. Har man arbejdet iterativt, præsenterer man blot det endelige resultat i rapporten. Ønsker man at diskutere hvordan et resultat er fremkommet sprint-for-sprint, kan dette gøres i diskussionsafsnittet.

Indhold

1 Introduktion **7**
- 1.1 Baggrund . . . 7
- 1.2 Eksisterende arbejde . . . 7
- 1.3 Systemskitse . . . 7
- 1.4 Problemformulering/Hypotese . . . 7

2 Udviklingsproces og metode **8**
- 2.1 Udviklingsproces . . . 8
- 2.2 Udviklingsmetoder . . . 8
- 2.3 Risikohåndtering . . . 9
- 2.4 Brug af AI . . . 9
  - 2.4.1 Uddelegering af arbejde . . . 9
  - 2.4.2 Fremgangsmåde . . . 10
  - 2.4.3 Bedømmelse af genererede output . . . 10
  - 2.4.4 Ansvarlighed og transparens . . . 10

3 Brugerkrav **11**
- 3.1 Introduktion . . . 11
- 3.2 Systemkontekst og aktører . . . 11
- 3.3 Funktionelle krav . . . 11
  - 3.3.1 Elicitering og analyse . . . 11
  - 3.3.2 Specifikation . . . 11
- 3.4 Ikke-funktionelle krav . . . 12
  - 3.4.1 Elicitering og analyse . . . 12
  - 3.4.2 Specifikation . . . 12
- 3.5 Systemets domænemodel . . . 13

4 Systemarkitektur **15**
- 4.1 Introduktion . . . 15
- 4.2 Systemets ydre grænser og relationer . . . 15
- 4.3 Analyse af arkitektur . . . 16
- 4.4 Systemets struktur og grænseflader . . . 17
  - 4.4.1 Interaktion mellem delsystemer . . . 19
  - 4.4.2 Interfaces og protokoller . . . 19
  - 4.4.3 Datamodeller og persistering . . . 20

5 Design og implementering **22**
- 5.1 Introduktion . . . 22
- 5.2 Delsystem <delsystem navn> . . . 22
  - 5.2.1 Introduktion . . . 22
  - 5.2.2 Design og implementering af <sw komponent navn> . . . 22
  - 5.2.3 Design og implementering af <hw komponent navn> . . . 23
  - 5.2.4 Design og implementering af <database komponent navn> . . . 23

5.2.5 Tests og resultater . . . 23
5.2.6 Delkonklusion . . . 23

6 Integrationstest **27**
6.1 Introduktion . . . 27
6.2 Teststrategi . . . 27
6.3 Integrationstest af use case 1 . . . 27
6.3.1 Opstilling . . . 27
6.3.2 Resultater . . . 27
6.4 Diskussion . . . 27
6.5 Delkonklusion . . . 27

7 Brugeraccepttest **29**
7.1 Introduktion . . . 29
7.1.1 Opstilling . . . 29
7.1.2 Resultater . . . 29
7.2 Diskussion . . . 29
7.3 Delkonklusion . . . 29

8 Diskussion **31**
8.1 Problemstilling og projektets løsning . . . 31
8.2 Proces og metode . . . 31
8.3 Brugerkrav . . . 31
8.4 Arkitektur . . . 31
8.5 Design . . . 31

9 Konklusion **33**
9.1 Fremtidigt arbejde . . . 33

Bibliografi **34**

10 Appendiks **35**
10.1 Krav til rapport og supplerende materiale . . . 35
10.1.1 Antal tegn . . . 35
10.1.2 Supplerende materiale . . . 35
10.2 Sidebudget for rapport . . . 37
10.3 Relevante rapportsektioner og diagrammer . . . 38
10.3.1 Software Engineering-projekt . . . 38
10.3.2 Embedded Software Engineering-projekt . . . 39
10.3.3 Electronics Engineering-projekt . . . 40
10.4 Code Listing Examples . . . 41
10.5 Table examples . . . 42
10.6 Rotated Full-page Image Example . . . 43

Figurer

1 Rapportstruktur baseret på V-Modellen 2

3.1 Eksempel: domænemodel 14

4.1 Eksempel: et detaljeret context diagram 16
4.2 Eksempel: arkitektur (package diagram) 18
4.3 Eksempel: alokering af arkitektur (deployment diagram) 18
4.4 Eksempel: systemsekvensdiagram 19
4.5 Eksempel: protokol (Skænkningsinterface) 20
4.6 Eksempel: protokol (REST sale POST) 20
4.7 Eksempel: protokol (REST sale Response) 20

5.1 Eksempel: klassediagram 23
5.2 Eksempel: sekvensdiagram 24
5.3 Eksempel: statediagram 24
5.4 Eksempel: Intern blokdiagram 25

10.1 Figure Example full-page landscape 43

Tabeller

5.1 Eksempel: Resultater af enhedstest for backend 26
5.2 Eksempel: Resultater af intern integrationstest i backend 26

6.1 Eksempel: Resultat af integrationstest mellem systemets delsystemer 28

7.1 Eksempel: Resultater fra brugeraccepttest 30

10.1 Sidebudget for rapport 37
10.2 Relevant sections and diagrams for Software Engineering Projects 38
10.3 Relevant sections and diagrams for Embedded Software Engineering Projects 39
10.4 Relevant sections and diagrams for Electronics Engineering Projects 40
10.5 Table example with footnote 42

Kodeudsnit

10.1 Unboxed code listing 41
10.2 Boxed Code Listing 41
10.3 Bash Listing Example 41
10.4 Boxed Side-by-Side Code Listing 41
10.5 Boxed Two Row Code Listing 42

Kapitel 1

Introduktion

Dette kapitel introducerer baggrund og motivation for projektet og skitserer det problem, der behandles i denne rapport. Det præsenterer relevant eksisterende arbejde og giver et overordnet konceptuelt overblik over det foreslåede system for at støtte læserens forståelse. Endelig defineres problemformuleringen, der fastlægger projektets omfang og mål. (2-4 sider)

1.1 Baggrund

Her introduceres projektets kontekst og motivation. Afsnittet beskriver domænet, relevante tendenser og det underliggende problemområde, der skaber behovet for en løsning. Formålet er at give læseren tilstrækkelig kontekst til at forstå, hvorfor projektet er relevant.

1.2 Eksisterende arbejde

Her gennemgåes relevante eksisterende løsninger, forskning mm. inden for domænet. Afsnittet opsummerer eksisterende fremgangsmåder, disses styrker og begrænsninger og placerer projektet i forhold til disse. Formålet er at demonstrere indsigt i state-of-the-art og begrunde behovet for den foreslåede løsning.

1.3 Systemskitse

Her præsenteres en overordnet konceptuel beskrivelse af det foreslåede system. Afsnittet skitserer hovedideen, nøglekomponenterne og hvordan systemet adresserer det identificerede problem. Formålet er at give læseren en intuitiv forståelse af løsningen før den detaljerede analyse. Det anbefales at vise en figur, f.eks. et rigt billede. Hold det funktionelt og undgå detaljer som vi på dette tidspunkt ikke har viden om. F.eks. teknologi som Raspberry Pi, React mm.

1.4 Problemformulering/Hypotese

Her defineres det problem, projektet adresserer. Afsnittet angiver omfang, mål og eventuelle antagelser eller begrænsninger. Problemformuleringen bør være klar, præcis og fungere som fundament for de efterfølgende krav, for design og evaluering. Afsnittet bør basere sig direkte på `Baggrund` *og* `Eksisterende arbejde`*.*

Kapitel 2

Udviklingsproces og metode

Dette kapitel beskriver den valgte projektmodel og de udviklingsmetoder, der styrer analyse, design, implementering og test af systemet. Formålet er at sikre en struktureret og systematisk udviklingsproces. (1-2 sider)

2.1 Udviklingsproces

Her beskrives den valgte projektmodel, herunder strategien for at bevæge sig fra den indledende idé til det endelige resultat, og hvordan modellen er tilpasset projektets kontekst. Eksempler kan være stage-gate, vandfald, iterativ, spiral, agil eller hybride modeller.

Eksempel: Udviklingsproces

Baseret på projektets formulering og kontekst er dette projekt karakteriseret ved følgende:

- Brugerinteraktion med systemet er afgørende og skal adresseres tidligt i processen
- Domænet er ikke fuldt forstået og skal undersøges tidligt for at klarlægge, hvad der reelt kræves

For at imødekomme dette, anvendes en stage-gate-model inspireret af UP, da den tillader os at styrke fokus på domæneundersøgelse i starten af projektet og derefter fokusere på iterativ udvikling. Følgende faser vil blive brugt:

FASE I: Informationsindsamling og Interviews. I denne fase undersøges eksisterende litteratur og der udvælges en række interessenter...

FASE II: Kravanalyse og arkitekturudkast. ...

FASE III: Delsystemudvikling, integration og test. ...

FASE IV: Indkøring og overlevering. ...

2.2 Udviklingsmetoder

Her beskrives de specifikke udviklingsmetoder og teknikker, der anvendes for at nå projektets mål. Afsnittet kan inkludere metoder til requirements engineering (f.eks. FURPS+), udvikling (f.eks. UML), undersøgelse (f.eks. litteratursøgning), koordinering (f.eks. Task board) og mere.

Eksempel: Udviklingsmetoder

Til projektets faser anvendes følgende metoder:

...

Fase II: Krav Funktionelle krav beskrives ved hjælp af Use Cases, og ikke-funktionelle krav udledes ved hjælp af FURPS [ref]. I forbindelse med kravene modelleres systemet i brugerens domæne med en domænemodel.

Fase II: Arkitekturudkast Den indledende systemarkitektur, bygges på basis af Domænemodellen og modelleres med UML komponent- og deployment-, og systemsekvensdiagrammer, således at delsystemer kan bygges på basis af arkitekturen.

FASE III: Delsystemudvikling og test. I denne fase anvendes en iterativ arbejdsmetode til implementering af komponenter indenfor det ekelte delsystem. Parallelle arbejdsprocesser understøttes ved hjælp af Versionsstyring, Git, og feature-branches ...

...

2.3 Risikohåndtering

Her beskrives hvordan risici og prioritering af krav håndteres gennem projektets levetid. Dette kan f.eks. være ved hjælp af en risikomatrice, der benyttes til at prioritere krav i løbet af projektet. Risici kan være af forskellig karakter: Arbejdsmæssige, kravsmæssige (er krav F1.2 er mulig at opfylde eller måle?) og tekniske. Tekniske risici kan eksistere både på arkitektur- (kan arkitekturen rumme projektets løsning?) og på designniveau (kan valgte implementering opfylde performance krav?). Risikomatrice og kravsprioritering håndteres typisk i et separat dokument/tabel/etc. som opdateres løbende i projektets forløb. Krav kan prioriteres nummerisk eller ved hjælp af MoSCoW.

Eksempel: Risikohåndtering

Identificerede risici dokumenteres og vurderes i en risikomatrice, der vedligeholdes i et regneark og løbende opdateres i projektet. Særlig opmærksomhed rettes mod arkitekturkritiske risici, da de har direkte indflydelse på centrale designbeslutninger. De fremhævede arkitektur-risici anvendes i Kapitel 4 som baggrund for valg af systemarkitektur.

2.4 Brug af AI

Brugen af AI beskrives ved hjælp af 4 metrikker baseret på Framework for AI Fluency. Der forefindes undervisningmateriale (videoer mm.) omkring frameworket på denne side: https://aifluencyframework.org/. Indholdet fra dette afsnit kan kopieres ind i AU's AI Deklaration, som SKAL afleveres som bilag til projektet, såfremt AI har været anvendt.

2.4.1 Uddelegering af arbejde

"Delegation refers to the ability to identify when and how to use AI tools and modalities effectively in creative and problem-solving processes. It involves understanding the capabilities and limitations of various AI technologies and making informed decisions about when to use AI for automation, augmentation, or independent agent-mediated experiences."[1]

2.4.2 Fremgangsmåde

"Description encompasses the skills needed to effectively communicate ideas, requirements, constraints, and other aspects of creative visions to AI systems. It involves crafting clear, specific, and well-structured prompts (using a wide range of prompting techniques) and other elements that guide and enable AI tools to produce desired behaviors and outputs."[1]

2.4.3 Bedømmelse af genererede output

"Discernment involves the critical evaluation of AI-generated outputs, understanding their quality, relevance, potential biases, and other salient characteristics. It also includes the ability to iterate and refine the collaborative process with AI tools."[1]

2.4.4 Ansvarlighed og transparens

"Diligence refers to the responsible use of AI, including ethical considerations, transparency about AI use, and taking accountability for the final products created with AI assistance."[1]

Eksempel: Brug af AI

Uddelegering af arbejde AI bruges til at effektivisere opgaver, hvor gruppen allerede har viden, men begrænses på områder, hvor læring er et mål i sig selv.
- Backend: AI bruges primært til inspiration til den indledende arkitektur. Gruppen står selv for implementering, tilpasning og omstrukturering.
- Frontend: Da GUI ikke er et centralt læringsmål, bruges AI i høj grad til udvikling af brugergrænsefladen.

Fremgangsmåde Der formuleres detaljerede prompts og der stilles opklarende spørgsmål. Antagelser minimeres og AI promptes eksplicit til at beskrive hvert trin i sin proces. Dette giver bedre kontrol over retningen og gør det muligt at opdage misforståelser, før der genereres output.

Bedømmelse af genererede output AI'ens arbejdsproces evalueres løbende, ligesom det endelige resultat. Output skal kunne forklares og forstås, før det anvendes.
Samtalerne holdes korte og nulstilles om nødvendigt undervejs for at undgå hallucinationer og fejlagtige output. Dette kan opstå ved lange samtaler.

Ansvarlighed og transparens Der anvendes flere AI-værktøjer:
- Claude Code: Bruges til kodearbejde direkte i terminalen.
- Gemini: Bruges til inspiration til den indledende struktur.
- Figma: Bruges til hurtig GUI-prototyping.
- Claude Design: Bruges til videreudvikling af GUI efter import af Figma-designs.

Der prioriteres sikker og ansvarlig AI-anvendelse. AI får ikke adgang til følsomme filer som .env, og alt genereret indhold gennemgås og valideres, før det anvendes.

Kapitel 3

Brugerkrav

Dette kapitel beskriver kravene til systemet fra brugerperspektiv. Det indleder med at præsentere konteksten for systemet og dets brugere. Herefter udledes, analyseres og specificeres de funktionelle og ikke-funktionelle krav. Endelig indeholder kapitlet en model af systemet beskrevet i brugerens termer. Dette kapitel bør være læsbart for en ikke-teknisk domæneekspert. Rapporten bør præsentere udvalgte krav og henvise til et separat kravspecifikationsdokument for resten. Hvis krav eller prioriteter har ændret sig i løbet af projektet, kan dette diskuteres i diskussionskapitlet. Husk at kravene skal være specifikke nok til at man kan udvikle en løsning på basis af disse, men ikke så specifikke, at der tages beslutninger om teknologi og design, med mindre at eksterne krav dikterer dette. (2-4 sider)

3.1 Introduktion

3.2 Systemkontekst og aktører

Dette afsnit beskriver systemets kontekst fra et bruger- og domæneperspektiv. Det identificerer aktørerne, der interagerer med systemet, og det miljø, som systemet opererer i. Formålet er at forstå, hvem der bruger systemet, og hvem systemet interagerer med. Et aktør-kontekstdiagram kan være nyttigt her.

Eksempel: Systemkontekst og aktører

Systemet skal kunne opstilles ved ikke-betjente udskænkningssteder, f.eks. i cafeteriet i den lokale sportshal. Systemet har to aktører:

Kunden kan således forventes at være i alderen 15-99 år, selvstændig, og i stand til at betille og betale en drik i almindelig betjent café.

Administrator som redigerer drikke og vedligeholder systemet

3.3 Funktionelle krav

3.3.1 Elicitering og analyse

Funktionelle krav udledes ved at analysere interaktionerne mellem aktører og systemet. Forklar hvordan kravene er udledt (f.eks. interview), og vis en oversigt (diagram eller korte beskrivelser).

3.3.2 Specifikation

På basis af analysen beskriver systemets adfærd fra et brugerperspektiv. Kravene struktureres som Use Cases, User stories, eller på listeform. Kravene bør være nummererede.

Eksempel: Funktionelle krav - Elicitering og analyse

Den primære aktør, brugeren, forventes i stand til at betille og betale en drik i almindelig betjent café. Bestillingsflow skal således minde om det som man kender fra betjente caféer: Vis menukort, vælg drik, betal og modtag bestilling. Da systemet skal køre autonomt, er det vigtigt at det løbende monitorerer bestilling og opskænkning, og agerer på anomali. F.eks at opfyldning stopper straks, hvis glasset fjernes eller overfyldes.
Specifikation

UC1: Bestil drik En kunde tilgår Drinkbot 4000 via webklienten og får vist en liste over tilgængelige drikke. Kunden vælger en drik og afgiver en bestilling, hvorefter den samlede pris vises. Kunden identificerer sig ved at føre sin mobiltelefon med aktivt kreditkort hen til RFID-læseren. Systemet validerer betalingen hos banken, og ved godkendelse igangsættes opskænkning af den valgte drik.

UC2: Skænk drik Systemets vægt måler vægten af det tomme glas og aktiverer den relevante doseringsenhed. Drik påfyldes, indtil den ønskede mængde er opnået baseret på løbende vægtmålinger, hvorefter opskænkningen stoppes.

3.4 Ikke-funktionelle krav

3.4.1 Elicitering og analyse

*Ikke-funktionelle krav identificeres ud fra kvalitetskrav og begrænsninger i problemdomænet. Disse omfatter illities som f.eks. ydeevne, skalerbarhed og pålidelighed, der påvirker systemets design (f.eks. Kameraet skal filme persons gang (2 m/s) ad 10 meter sti => krav: filmklip længde skal være minimum 5 sekunder. Der må maksimalt være fejl i 1 sekund af visning af en 2-timers film => Oppetid = 1 - 1/(2*60*60) = 99.99% af tiden).*

3.4.2 Specifikation

De konkrete krav. Kan være grupperede, og skal være nummererede.

Eksempel: Ikke-funktionelle krav - Elicitering og analyse

Systemet skal kunne opstilles f.eks. i cafeterier i den lokale sportshal. Ved arrangementer med 200 deltagere skal alle kunne få skænket en drik hver time. En drik er typisk 0,5 liter, dvs. den skal kunne servere 100 liter i timen og håndtere 200 transaktioner. Tidsbudgettet ser således ud:

$$T_{kunde} = T_{betjening} + T_{kortvalidering} + T_{sknkning} \tag{3.1}$$

$$T_{kunde,max} = \frac{60[min]}{200[pers]} = 0,3[min] = 20[sek] \tag{3.2}$$

$$T_{sknkning} = T_{kunde} - T_{betjening} + T_{kortvalidering} \tag{3.3}$$

$$Antages: \tag{3.4}$$

$$T_{betjening} = 12[s], T_{kortvalidering} = 5[s] \tag{3.5}$$

$$T_{sknkning} = 20 - 12 - 5 = 3[sek] \tag{3.6}$$

Specifikation

NF1: Brugervenlighed 9 ud af 10 slutbrugere skal kunne betjene systemet på 12 sekunder (+/- 1 s), med henblik på at få leveret en drik.

NF2: Performance Systemet skal validere kreditkort indenfor 5 sekunder, antaget at eksternt pengeinstitut over holder standard XYZ.

NF3: Performance Systemet skal kunne skænke en 0,5 liter drik indenfor 3 sekunder.

NF4: Performance Drik skal skænkes med en præcision på 2 ml (+/- 0.5 ml).

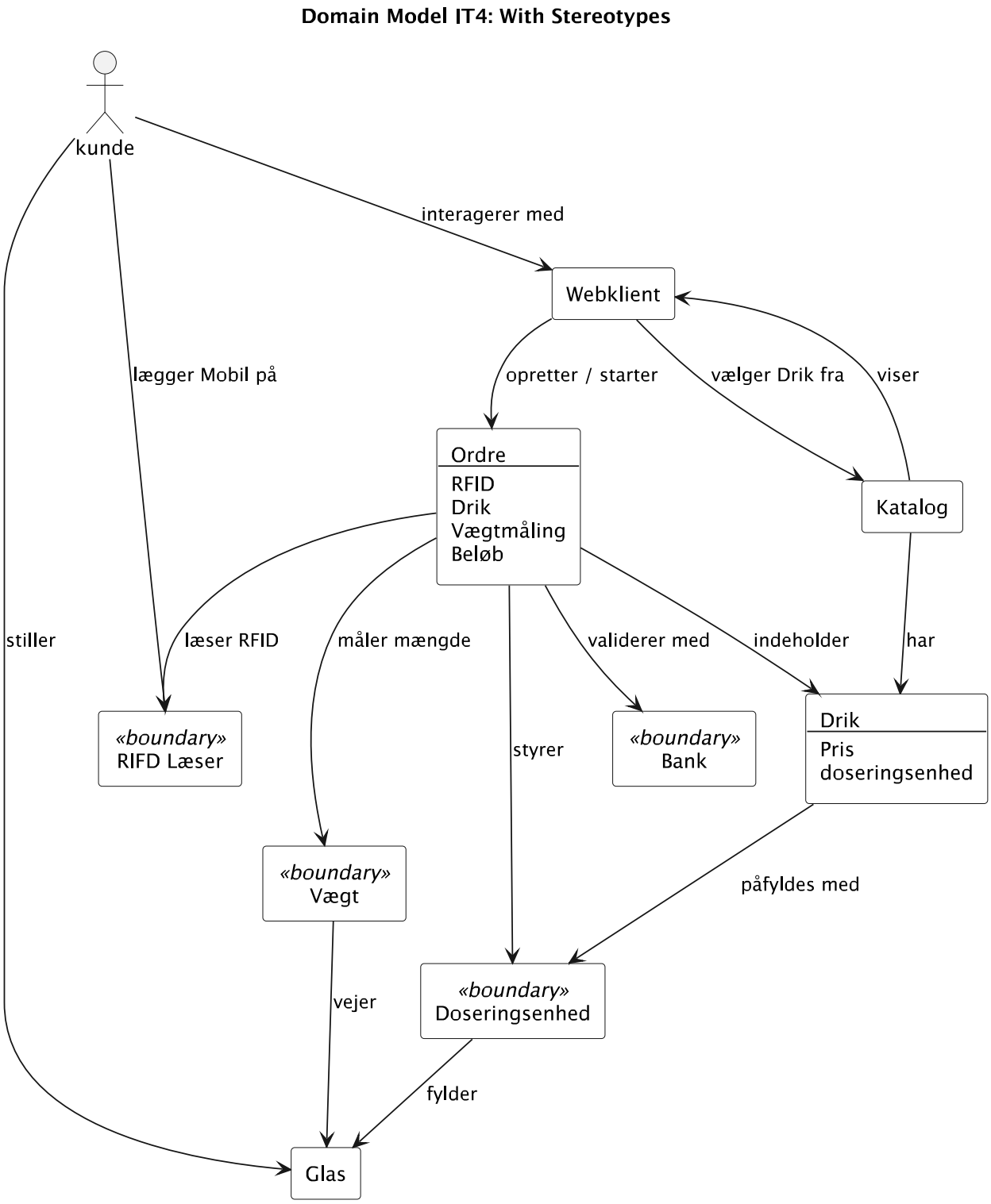
...

3.5 Systemets domænemodel

Dette afsnit beskriver systemets funktionelle enheder og deres relationer fra et brugerperspektiv. Det er den sidste brugerorienterede del. Herefter bliver det teknisk i de følgende kapitler. En UML-domænemodel er nyttig her. Husk at domænemodellen udledes direkte fra kravene og beskriver funktionalitet, data og ydre grænser for systemet som helhed, men at den ikke forholder sig til teknologi og interne interfaces! Det kommer først i arkitektur og design!

Eksempel: Domænemodel

Systemets domænemodel i Figur 3.1 viser systemets vigtigste entiteter, fysiske såvel som abstrakte. Det er valgt at `Ordre` både har ansvar for den valgte drik og for selve skænkningen.



Figur 3.1: Eksempel: domænemodel

Kapitel 4

Systemarkitektur

Dette kapitel præsenterer systemets overordnede arkitektur fra et strukturelt perspektiv. Det definerer først systemets grænser og relationer til eksterne systemer, hvorefter den interne struktur og grænseflader beskrives sammen med centrale teknologivalg. (2-4 sider)

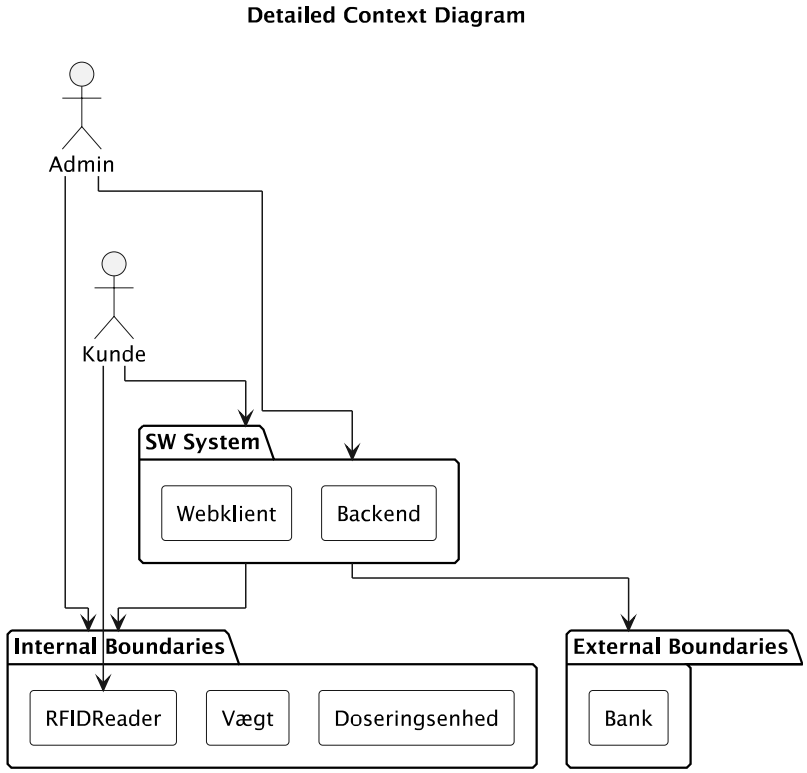
4.1 Introduktion

4.2 Systemets ydre grænser og relationer

Dette afsnit identificerer, hvad der er systemet, hvor dets grænser går og hvad dets eksterne relationer er. Dette er de ydre grænse for den arkitektur, som vi giver flere detaljer i de følgende afsnit. Relevante diagrammer i dette afsnit kan være et simpelt, overordnet UML Package-diagram baseret på domænemodellen, der fremhæver systemet og dets grænser, eller et System Context Diagram, hvis C4 anvendes.

Eksempel: Systemets ydre grænser og relationer

Systemets kontekst er præciseret i Figur 4.1. Kunde og Admin tilgår systemets software via en Webklient, men har også begge to behov for at tilgå de tilsluttede hardware komponenter. Software systemet tilgår Banken via sin Backend Service.



Figur 4.1: Eksempel: et detaljeret context diagram

4.3 Analyse af arkitektur

Dette afsnit idenftificerer arkitektur-kritiske riscisi og foreslår arkitekturelle løsninger som reducerer disse. Identifikationen af riscisi kan basere sig på at (a) finde ikke-funktionelle krav, som den indledende domænemodelbaserede arkitektur ikke opfylder, og (b) identificere arkitekturkritiske teknologier (softwareframeworks, grænseflader mm.). Kritiske risici reduceres ved tilpasning af arkitekturen, grænseflader eller ved valg af teknologi. Analysen indeholder typisk argumenter baseret på litteratur eller små eksperimenter (dokumenteret i bilag). Den resulterende arkitektur præsenteres i det næste afsnit

Eksempel: Analyse af arkitektur

Skænkningshastighed
De ikke-funktionelle krav `NF3` om hurtig opskænkning og `NF4` om præcision, stiller krav til reguleringsloopet ved dosering af drikken. For at kunne understøtte disse, skal reguleringssløjfen kunne reagere indenfor kort tid:

$$T_{sknkning,max} = 3[sek] \tag{4.1}$$

$$V_{fejl,max} = 0,002[l] \tag{4.2}$$

$$Flow_{min} = \frac{0,5[l]}{3[s]} = 0,17[l/s] \tag{4.3}$$

$$T_{it,max} = \frac{0,002[l]}{0,17[l/s]} = 12[ms] \tag{4.4}$$

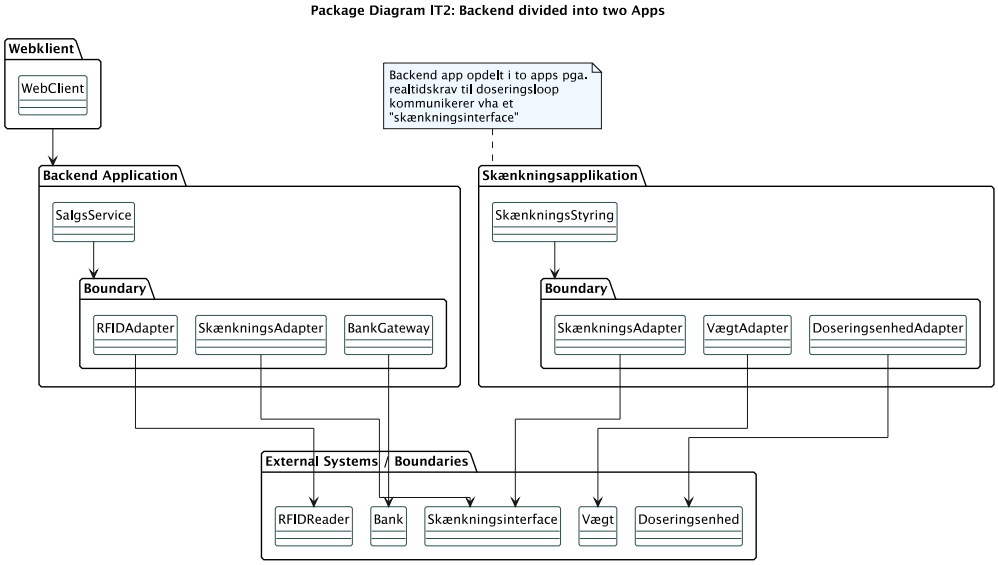
En multithreaded Linux applikation på en Raspberry Pi vil have svært ved at imødekomme dette krav og vil dermed udgøre en høj teknisk risiko. Forsøg med en microcontroller uden OS har vist gode resultater (Bilag X), og systemet opdeles derfor i to delsystemer, Backend og Skænkning, hvor den sidste del afvikler den tidskritiske regulering af doseringen. Den tekniske risiko reduceres herved til et acceptabelt niveau.

4.4 Systemets struktur og grænseflader

Her præsenteres den valgte arkitektur, som resultat af analysen i forrige afsnit. Arkitekturen beskriver systemets overordnede struktur, og de delsystmer som den består af. Den suppleres med beskrivelser af grænseflader, protokoller og datamodeller, der deles af delsystemerne. Med viden om den overordnede struktur, fælles datamodeller og hvordan der kommunikeres mellem delsystmerne, har vi nu et solidt grundlag for at udvikle delsystemerne i næste kapitel. Nyttige diagrammer er lagdelte UML Package-diagrammer, UML Deployment-diagram eller et C4 Container-diagram.

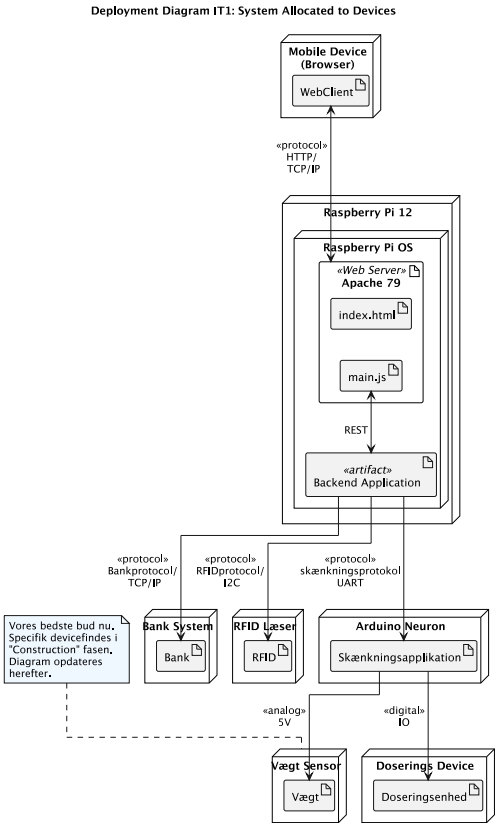
Eksempel: Systemets struktur og grænseflader

Systemets arkitektur er baseret på inputtene fra vores analyse af de ikke-funktionelle krav. Kravene til opskænkningshastighed har resulteret i en distribueret løsning med to separate delsystemer, som vist i Figur 4.2.



Figur 4.2: Eksempel: arkitektur (package diagram)

Delsystemerne deployeres som vist i Figur 4.2, således at `Backend applikation` og `Webserver` afvikles på en Linux-baseret embedded enhed, skænkningsapplikationen afvikles på en microcontroller og endelig afvikles `Web Client` i en browser på brugerens PC eller mobile enhed. Detaljer om designvalg kan findes i de respektive designafsnit i kapitlerne X.x og Y.y.



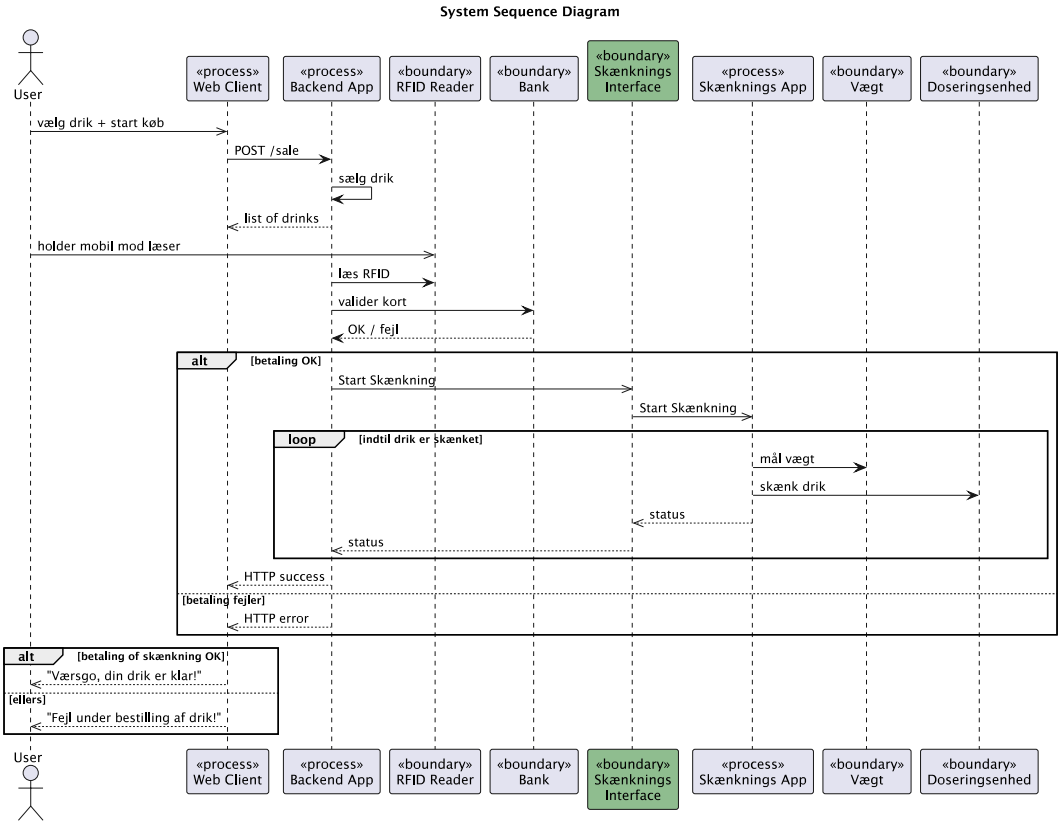
Figur 4.3: Eksempel: alokering af arkitektur (deployment diagram)

4.4.1 Interaktion mellem delsystemer

Her beskrives, hvordan delsystemerne interagerer og udveksler data under drift. Gerne med et UML System Sequence-diagram for hver relevant Use Case / User Story.

Eksempel: Interaktion mellem delsystemer

Med udgangspunkt i UC1 beskriver Figur 4.4 interaktionen mellem de to delsystemer og grænseflader. `Backend App` og `Skænknings App` kommunikerer via `Skænkningsinterface` som er markeret med grøn.



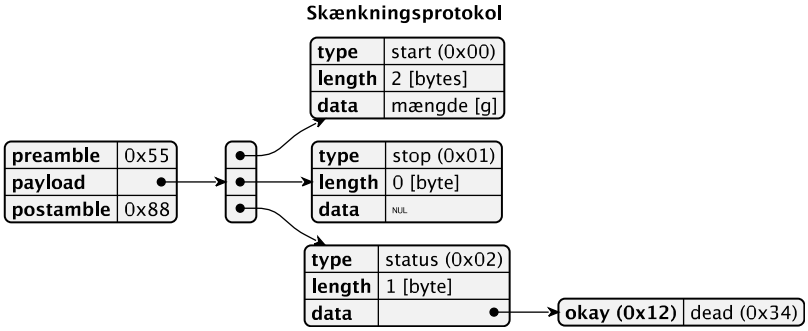
Figur 4.4: Eksempel: systemsekvensdiagram

4.4.2 Interfaces og protokoller

Her beskrives detaljer om de grænsefladeteknologier og protokoller, som er nævnt i UML Deployment-diagram eller C4 Container-diagram.

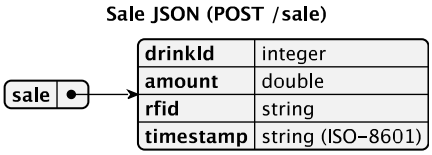
Eksempel: Interfaces og protokoller

Skænkningsinterface
På basis af analysen benytter dette interface et UART fysisk lag, med en baud rate på 115200 og en protokol som beskrevet i Figur 4.5.

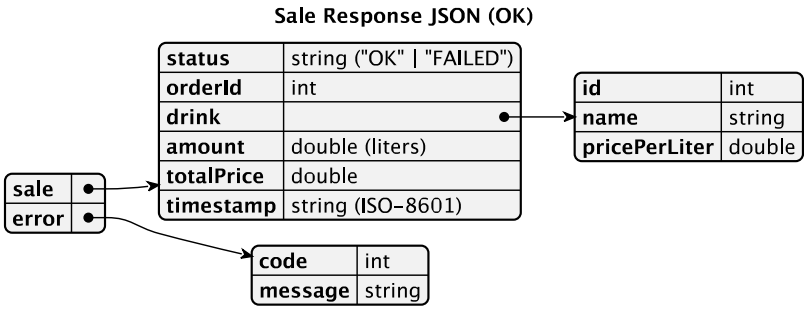


Figur 4.5: Eksempel: protokol (Skænkningsinterface)

Webinterface
Interfacet mellem `Webclient` og `Backend` benytter sig af REST over HTTPS. Ved et salg sendes en HTTP/POST fra `Webclient`, hvorpå der modtages en response fra `Backend`. Formatet af JSON beskederne er vist i henholdsvis Figur 4.6 og Figur 4.7.



Figur 4.6: Eksempel: protokol (REST sale POST)



Figur 4.7: Eksempel: protokol (REST sale Response)

4.4.3 Datamodeller og persistering

Hvis relevant beskriver dette afsnit data modeller, persisteringsmetoder, og teknologier som anvendes på tværs af arkitekturen (f.eks. SQL/NoSQL/filbaseret). Relevante diagrammer kan f.eks. være et ER-diagram.

Eksempel: Datamodeller og persistering

Logging
Logs gemmes som tekstfiler og anvender følgende format:

```
dd:mm:åå tt:mm:ss - <log niveau> - <klassenavn::funktion> - <besked>
```

Kapitel 5

Design og implementering

Dette kapitel beskriver design og implementering af systemets delsystemer. Det præciserer hvordan indholdet i arkitekturen realiseres gennem software- og/eller hardwarekomponenter, herunder designbeslutninger, modeller og implementeringsdetaljer. Kapitlet danner grundlaget for den efterfølgende test og evaluering af systemet. Bemærk at kapitlet bruger en top-down approach, hvor hvert delsystem fra arkitekturen beskrives i separate afsnit, som der efterfølgende bores ned i og åbnes op. Hvis et delsystem har flere undernieauer (F.eks. Container-Component-Class), kan disse om nødvendigt bores ned i gennem yderligere underafsnit. (4-10 sider)

5.1 Introduktion

5.2 Delsystem <delsystem navn>

5.2.1 Introduktion

Dette afsnit introducerer et specifikt delsystem og de komponenter, det består af. I det følgende er givet eksempler på indhold til komponenter af forskellig type. Vær opmærksom på at du alene skal designe dette delsystems interaktion med andre delsystemer udfra det som du kan læse i arkitekturen! Dvs. ingen detaljer om andre delsystemer i dette afsnit!!

5.2.1.1 Struktur af delsystem

Her viser du hvad som delsystemet består af og hvordan det er internt forbundet. Beskrivelsen skal kunne stå alene og må ikke indeholde komponenter og logik fra andre delsystemer, vi kender kun interfacet til dem (via arkitekturen). For software-delsystemer med flere abstraktionsniveauer kan du bruge et C4 Component Diagram eller et UML Component Diagram og bore ned i disse i underafsnit. For et simplere system med færre abstraktionsniveauer kan du nøjes med et UML Class Diagrams. Hardware kan præsenteres ved hjælp af et overordnet SysML Internal Block Diagram, eventuelt med visning af software allokering.

5.2.1.2 Interaktion mellem komponenter

Her vises hvordan at delsystements komponenter interagerer, gerne med udgangspunkt i Use Cases/User Stories, og ved hjælp af UML Sequence Diagrams. Til software, der er mere dataflow-orienteret, f.eks. algoritmer, functional programs eller DSP-kode, som ikke er objektorienteret, kan du benytte UML Activity Diagrams.

5.2.2 Design og implementering af <sw komponent navn>

Afsnittet introducerer en specifik SW komponent, dens design, implementering og enhedstest. Nyttige diagrammer her er UML Class Diagrams, UML Sequence Diagrams, UML State Diagrams, UML Activity Diagrams.

5.2.3 Design og implementering af <hw komponent navn>

Afsnittet introducerer en specifik HW komponent, dens design, implementering og enhedstest, herunder beregninger, PCB Layout, EMC målinger mm. Nyttige diagrammer her er UML State Diagrams, Hardware Schematic Diagram.

5.2.4 Design og implementering af <database komponent navn>

Eksempel på en specifik komponent type, som beskrives med specifikke modeller: Databaseskema (tabeller, PK/FK, begrænsninger), og som har specifikke overvejelser om fysisk design, indeksering, performanceovervejelser, ORM-modeller, migrationer

5.2.5 Tests og resultater

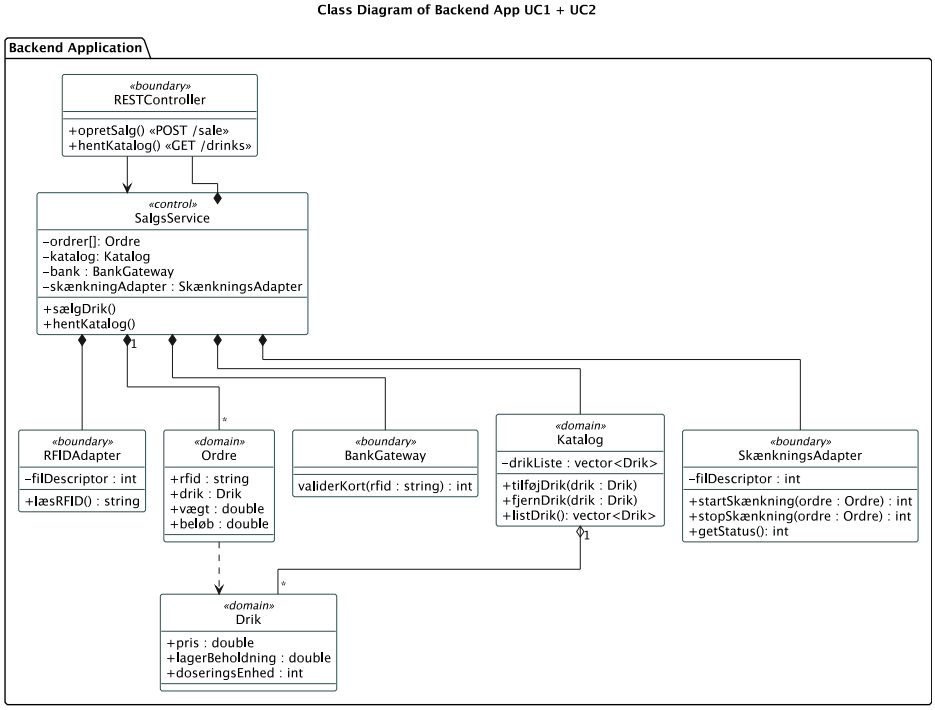
Resultater fra enhedstests, og integrationstests mellem komponenter, inklusive testmetrikker, hvis tilgængelige.

5.2.6 Delkonklusion

Eventuel opsummering på delsystem

Eksempel: Backend Delsystem

Introduktion
Backend delsystemet har ansvaret for afvikling af salg og kommunikation med Skænkningsenheden.
Struktur af delsystem
Det centrale ansvar for koordineringen mellem klasserne er placeret i `SalgsService`, hvilket fremgår af klassediagrammet i Figur 5.1. Håndteringen af REST-endpoints er placeret i en separat klasse, `RestController`, som kommunikerer med `SalgsService` via events. Denne opdeling reducerer koblingen mellem komponenterne og øger samhørigheden.

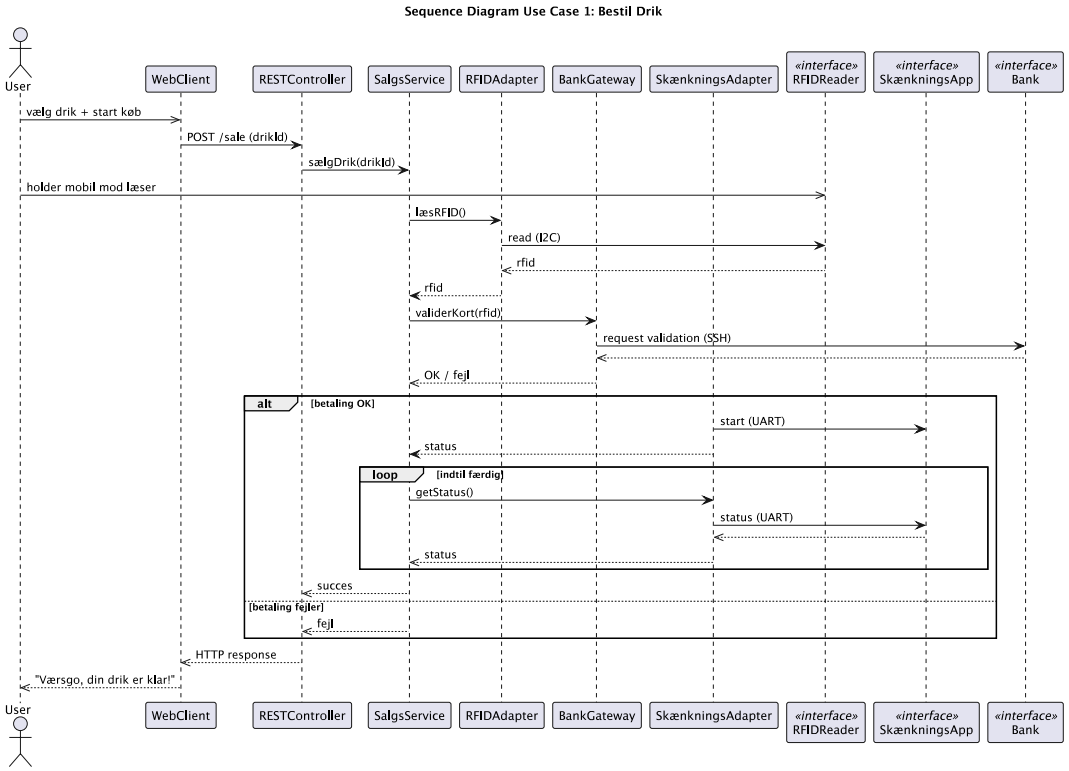


Figur 5.1: Eksempel: klassediagram

Eksempel: Backend Delsystem (fortsat)

Interaktion mellem komponenter

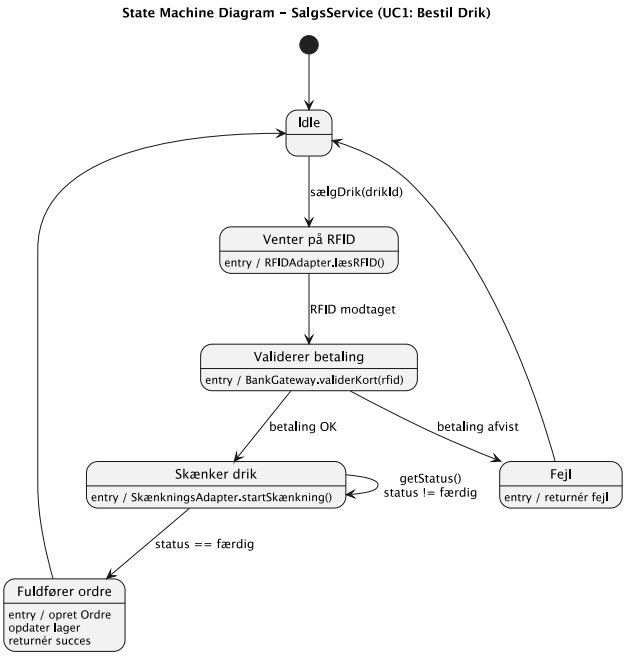
Som vist i Figur 5.2 interagerer `SalgsService` via adapterklasser med eksterne devices og aktører.



Figur 5.2: Eksempel: sekvensdiagram

Design og implementering af SalesService

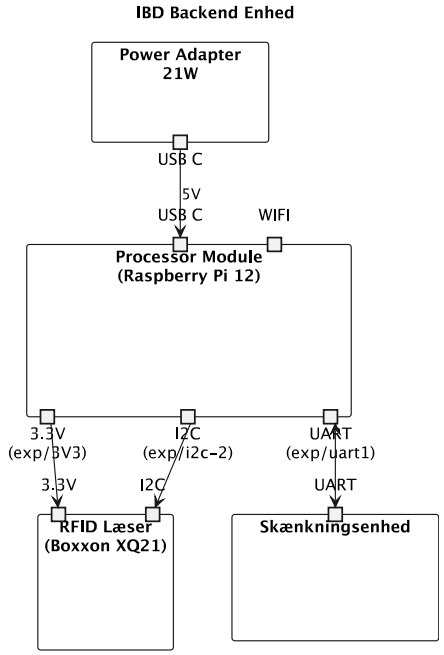
`SalesService` implementerer logikken beskrevet i `UC1` og `UC2`. Designet er baseret på state machinen vist i Figur 5.3.



Figur 5.3: Eksempel: statediagram

Eksempel: Backend Delsystem (fortsat)

Design og implementering af hardware Den overordnede sturktur for hardwaren er vist i Figur 5.4 og består af tre overordnede komponenter: Processor board, RFID læser og en strømforsyning.



Figur 5.4: Eksempel: Intern blokdiagram

Processorboard
Der er valgt en Raspberry Pi 12 fordi . . . (analyse her eller i bilag).
RFID Læser
Jfr. krav NF47, skal RFID læseren kunne læse kort af typen qi43 fra en afstand af maksimalt 3 centimeter. Ud fra undersøgelsen af læsere (bilag TT), er det valgt at anvende *Boxxon XQ21*, som opfylder givne krav og som kan tilsluttes Processorboardet via I2C.
Strømforsyning
Strømforsyningen er valgt ud fra krav (ref) til salgsområde og baseret på målinger af strømforbrug (Bilag ZZ). Målingerne viste at under maksimal belastning kan delsystemet bruge op mod 15 Watt. (ref) anbefaler desuden at anvende en 21 W power adapter. Strømforsyning xyz21 eller tilsvarende kan derfor anvendes.

Eksempel: Backend Delsystem (fortsat)

Tests og resultater
For at verificere implementeringen af backend-delsystemet er der udført enhedstests samt interne integrationstests mellem backend-komponenterne. Testene fokuserer på funktionaliteten i `SalgsService` og dets interaktion med de tilhørende boundary-klasser.

Enhedstest af SalgsService
Enhedstestene verificerer den centrale forretningslogik i `SalgsService`, herunder oprettelse af ordrer, håndtering af katalogdata og validering af salgsflowet. Eksterne afhængigheder såsom `BankGateway` og `SkænkningsAdapter` er erstattet med mock-objekter for at isolere testene til backend-logikken.

Test	Forventet resultat	Status
Opret salg med gyldig drik	Ordre oprettes korrekt	✓
Hent katalog	Liste af tilgængelige drikke returneres	✓
Salg med ugyldigt RFID	Salg afvises	✓
Salg ved afvist betaling	Skænkning initieres ikke	✓

Tabel 5.1: Eksempel: Resultater af enhedstest for backend

Intern integrationstest
Der er udført integrationstest mellem backend-komponenterne for at verificere, at de interne interfaces fungerer korrekt. Testene omfatter blandt andet kommunikationen mellem `RESTController` og `SalgsService` samt integration mellem `SalgsService`, `BankGateway` og `RFIDAdapter`.

Interface	Test	Status
RESTController → SalgsService	Salgsrequest videresendes korrekt	✓
SalgsService → RFIDAdapter	RFID-data modtages korrekt	✓
SalgsService → BankGateway	Valideringsresultat håndteres korrekt	✓
SalgsService → Katalog	Drikdata kan hentes korrekt	✓

Tabel 5.2: Eksempel: Resultater af intern integrationstest i backend

Testmetrikker
Der er gennemført i alt 8 automatiserede tests for backend-delsystemet. Alle tests bestod. Testdækningen for de centrale klasser i backend var:

- `SalgsService`: 92% line coverage
- `Katalog`: 100% line coverage
- `RESTController`: 85% line coverage

Delkonklusion
Backend delsystemet er opbygget med en Raspberry Pi 12, en XQ21 RFID læser, har et UART interface til Doseringssystemet og kommunikerer med Bankens API over Raspberry'ens WIFI forbindelse. Softwaren er centreret omkring SalgsService som håndterer salg og medierer information mellem Webklient og Doseringssystemet. Der er gennemført manuel end-to-end test med brug af simulerede udgaver af Bank API og Doseringssystem. Testresultaterne viser, at backendens centrale funktionalitet og interne komponentrelationer fungerer som forventet.

Kapitel 6

Integrationstest

Dette kapitel verificerer den valgte arkitektur og interaktionen mellem delsystemerne, med resultaterne af integrationstests. Der er tale om tekniske tests, som giver os sikkerhed for at interne krav er opfyldt. (1-2 sider)

6.1 Introduktion

6.2 Teststrategi

Beskrivelse af integrationstest strategi: Top-down, bottom-up, use-case drevet mm.

6.3 Integrationstest af use case 1

6.3.1 Opstilling

Beskrivelse af forudsætningerne for at gennemføre testen. Hvilken opsætning? Værktøjer?

6.3.2 Resultater

Beskrivelse af resultaterne?

6.4 Diskussion

Diskussion af resulaterne af integrationstestene

6.5 Delkonklusion

Hvad kan konkluderes ud fra resultaterne?

Eksempel: Integrationstest af delsystemerne Webclient, Backend og Skænkningssystem

Teststrategi
Integrationstesten fokuserer på kommunikationen mellem systemets tre primære delsystemer: Webclient, Backend og Skænkningssytemet. Interne integrationer i de enkelte delsystemer betragles som verificeret i tidligere testkapitler. Testene udføres med en top-down tilgang og verificerer de definerede interfaces mellem komponenterne.
Testopstilling
Testen udføres på den samlede hardwareplatform bestående af en Raspberry Pi med Backend Application og en Arduino med Skænkningsapplikationen. Webklienten anvendes som input til backendens REST-interface, mens kommunikationen mellem backend og skænkningsapplikationen foregår via det definerede skænkningsinterface over UART.
Resultater
Resultaterne i Tabel 6.1 viser at alle integrationstestene er genneført med godkendt resultat.

Interface	**Test**	**Resultat**
WebClient → Backend Application	REST-kald med salgs-request modtages og behandles	Godkendt
Backend Application → Skænkningsapplikation	Startkommando over skænkningsinterface modtages korrekt	Godkendt
Skænkningsapplikation → Backend Application	Statusmeddelelser returneres efter endt skænkning	Godkendt
Skænkningsapplikation → Doseringsenhed	Kommandoer til aktivering og stop videresendes korrekt	Godkendt
Backend Application → WebClient	Resultat fra backend returneres som HTTP-respons	Godkendt

Tabel 6.1: Eksempel: Resultat af integrationstest mellem systemets delsystemer

Diskussion
Integrationstesten verificerer, at kommunikationen mellem systemets delsystemer fungerer via de definerede interfaces. Testene viser, at Backend kan initiere en skænkning, modtage status fra Skænkningsapplikationen og kommunikere resultatet tilbage til webklienten. De interne funktioner i de enkelte delsystemer er ikke omfattet af denne test.
Delkonklusion
Integrationstesten viser at interfaces mellem delsystemerne fungerer som specificeret og at de definerede interfaces og protokoller er tilstrækkelige til de nuværende behov.

Kapitel 7

Brugeraccepttest

Dette kapitel beskriver valideringen af systemet i forhold til de brugerkravene. Det skitserer opstillingen af brugeraccepttesten og præsenterer resultaterne, idet det vurderes, om systemet opfylder sit tiltænkte formål. (1-2 sider)

7.1 Introduktion

7.1.1 Opstilling

Hvad er forudsætningerne for at gennemføre testen? Opsætning? Værktøjer?

7.1.2 Resultater

Hvad er resultaterne?

7.2 Diskussion

Diskussion af resulaterne af accepttestene

7.3 Delkonklusion

Hvad kan konkluderes ud fra resultaterne?

Eksempel: Brugeraccepttest af Drinkbot 4000

Introduktion
Brugeraccepttesten evaluerer systemet som en samlet løsning. Modsat tidligere enheds- og integrationstests fokuserer denne test på brugerens oplevelse og systemets evne til at opfylde de opstillede krav. Testen tager udgangspunkt kravene beskrevet i kapitel K.
Opstilling
Testen blev udført på den komplette Drinkbot 4000 prototype og udført sammen med en repræsentant fra YY. Tidsmålinger blev foretaget med en ekstern timer, mens doseringspræcisionen blev evalueret ved hjælp af vægtmålinger.
Resultater

Krav	Test	Resultat	Status
UC1	Bruger bestiller drik gennem webklient og gennemfører betaling	Bestilling gennemført	✓
UC2	Systemet skænker valgt drik baseret på vægtmåling	Drik skænkes korrekt	✓
NF1	10 brugere gennemfører en bestilling	9/10 indenfor 12 sekunder	(✓)
NF2	Tid fra RFID-identifikation til betalingsresultat måles	2,8 sekunder	✓
NF3	Tid for opskænkning af 0,5 liter måles	2,6 sekunder	✓
NF4	Doseringspræcision måles ved gentagne skænkninger	Afvigelse $\pm$0,4 ml	✓

Tabel 7.1: Eksempel: Resultater fra brugeraccepttest

Diskussion
Testresultaterne viser, at Drinkbot 4000 opfylder de opstillede brugerkrav. Brugeren kan gennemføre hele processen fra valg af drik til færdig opskænkning, og systemets performance ligger indenfor de specificerede grænser. De funktionelle krav blev verificeret gennem komplette brugerflows, mens de ikke-funktionelle krav blev evalueret gennem tidsmålinger og kvantitative målinger af doseringspræcision. Testen af `NF1` var begrænset af kun at være gennemført med en enkelt bruger og kan derfor ikke regnes for 100% gyldig.
Delkonklusion
Brugeraccepttesten viser, at systemet opfylder det tiltænkte formål som en automatisk drikkevareløsning. Alle testede krav blev opfyldt, og der blev ikke observeret fejl under gennemførelsen af de komplette brugerflows. Testen af `NF1` var dog begrænset af antallet af forskellige brugere. Resultaterne indikerer, at den valgte arkitektur og implementering understøtter de funktionelle krav samt de opstillede krav til brugervenlighed og performance.

Kapitel 8

Diskussion

Dette kapitel diskuterer projektets resultater. I forbindelse med et udviklingsprojekt udgør resultater såvel, krav, arkitektur, design og tests. Det giver derfor mening at diskutere bredere end blot at diskutere akcepttestresultater. Man kan diskutere oppe- eller nedefra. Man kan f.eks. diskutere problemstilling versus akcepttest, resultaterne af at anvende den valgte projektmodel og metode, resultaterne af at benytte den valgte arkitektur, resultaterne af designbeslutninger mm. Kapitlet diskuterer systemet i forhold til kravene og reflekterer over designbeslutninger, systemets ydeevne og begrænsninger. Diskussionen giver en kritisk vurdering af den samlede løsning. (1-3 sider)

8.1 Problemstilling og projektets løsning

Hvordan besvarer løsningen projektets problemstilling? Rammer den sit mål, eller er der gået teknik creep i den? Har I måtte justere undervejs?

8.2 Proces og metode

Diskutér hvordan det er gået med den valgte proces og metode. Hvad er hvad har fungeret godt for projektet og ikke? Har I måtte justere?

8.3 Brugerkrav

Diskutér kravene. Var de velformulerede og testbare? Hvordan er de blevet prioriteret? Har nogle vist sig urealistiske?

8.4 Arkitektur

Har den valgte arkitektur kunne rumme design og implementering? Har den begrænsninger? Har I måtte ændre den markant undervejs?

8.5 Design

Gav designvalg anledning til begrænsninger/muligheder? Er det holdbare valg som understøtte fremtidigt arbejde?

Eksempel: Diskussion

Dette kapitel diskuterer projektets samlede resultater. Diskussionen vurderer ikke kun, om systemet fungerer, men også hvilke konsekvenser de valgte løsninger har haft for udvikling, vedligeholdelse og fremtidig udvidelse.

Problemstilling og projektets løsning

Den udviklede løsning adresserer problemstillingen beskrevet i kapitel I ved at kombinere en webbaseret brugerflade med en backend-applikation og en separat skænkningsapplikation.

Under projektet blev der foretaget en afgrænsning af systemets funktionalitet. Eksempelvis blev fokus fastholdt på én central salgsproces fremfor at udvide systemet med yderligere brugerfunktionalitet. Dette har begrænset de tekniske riscisi og sikret fokus på de centrale krav.

En udfordring ved løsningen er dog, at den samlede brugeroplevelse afhænger af flere eksterne komponenter såsom betalingssystem, RFID-læser og doseringshardware. Dette betyder, at systemets kompleksitet primært ligger i integrationen mellem komponenterne frem for i den enkelte softwarekomponent.

Proces og metode

Den valgte projektmodel har fungeret hensigtsmæssigt i projektet. Den indledende analysefase gav mulighed for at identificere centrale risici, før implementeringen blev påbegyndt. Særligt opdelingen mellem backend og skænkningsapplikationen blev identificeret som en vigtig arkitekturbeslutning tidligt i processen.

Den iterative udviklingsproces i fase III gjorde det muligt løbende at afprøve designbeslutninger gennem implementering og test. Dette var særligt relevant for hardwareintegration, hvor den faktiske opførsel først kunne verificeres på den fysiske prototype.

En begrænsning ved processen var, at hardwareafhængigheder medførte ventetid i udviklingen af softwarekomponenter. En tidligere etablering af mocks eller simulatorer kunne have reduceret denne afhængighed.

Samlet set har den valgte proces understøttet projektets karakter, hvor både softwarearkitektur og fysisk integration skulle udvikles parallelt.

Brugerkrav

De funktionelle krav har fungeret godt som grundlag for både design og test. UC1 og UC2 beskriver tydeligt de centrale brugerinteraktioner og kunne direkte anvendes som udgangspunkt for sekvensdiagrammer og brugeraccepttest.

De ikke-funktionelle krav har ligeledes været vigtige for arkitekturen, især kravene relateret til performance og præcision. Kravene medførte eksempelvis behovet for at separere skænkningsapplikationen fra backend-applikationen.

Kravene har dog generelt været tilstrækkeligt konkrete til at understøtte designvalg og efterfølgende test.

Arkitektur

Den valgte arkitektur med en opdeling mellem backend og skænkningsapplikation har vist sig egnet til systemets krav. Opdelingen reducerer koblingen mellem forretningslogik og realtidsstyring, hvilket gør det muligt at udvikle og teste de to dele uafhængigt.

Under udviklingen viste det sig, at arkitekturen kunne rumme de nødvendige designændringer uden større omstrukturering. Den oprindelige opdeling mellem delsystemerne blev derfor bevaret gennem hele projektet.

En begrænsning ved arkitekturen er, at ændringer i kommunikationsprotokollen kræver koordinering mellem begge applikationer. Et mere avanceret kommunikationslag kunne i fremtiden reducere denne afhængighed.

Design

Flere designbeslutninger har haft betydning for systemets muligheder og begrænsninger. Anvendelsen af adapterklasser omkring eksterne enheder har bidraget til en lavere kobling og gjort det muligt at udskifte hardware uden at ændre den centrale forretningslogik.

De valgte designprincipper vurderes derfor som hensigtsmæssige for den nuværende løsning og giver mulighed for fremtidig udvidelse, eksempelvis flere drikke, flere doseringsenheder eller alternative betalingsmetoder.

Kapitel 9

Konklusion

Dette kapitel afslutter rapporten ved at opsummere de vigtigste fund og projektets bidrag. Det reflekterer over, hvordan problemformuleringen er blevet adresseret, og fremhæver de centrale resultater af arbejdet. (1-2 sider)

9.1 Fremtidigt arbejde

Dette afsnit skitserer åbninger for fremtidigt arbejde i form af potentielle forbedringer og udvidelser. Den identificerer begrænsninger i den nuværende løsning og foreslår retninger for videre udvikling og forfining.

Eksempel: Konklusion

Dette kapitel opsummerer projektets resultater og vurderer, hvorvidt den udviklede løsning opfylder projektets formål og opstillede krav. Afslutningsvis beskrives relevante muligheder for fremtidigt arbejde.

Konklusion

Projektets formål var at udvikle et automatiseret drikkevaresystem, der gør det muligt for en bruger at bestille en drik via en webklient, gennemføre betaling og få drikken automatisk opskænket. Der er udviklet en samlet løsning bestående af en webklient, en backend-applikation og en separat skænkningsapplikation.

Den valgte arkitektur har vist sig velegnet til systemets krav. Backend håndterer forretningslogikken, mens skænkningsapplikationen varetager den tidskritiske styring af doseringsprocessen. Denne opdeling har reduceret koblingen mellem systemets dele og gjort det muligt at udvikle og teste delsystemerne uafhængigt.

Gennem enhedstests, integrationstests og brugeraccepttest er det verificeret, at systemets centrale funktionalitet fungerer som forventet. Brugeraccepttesten viste, at løsningen opfylder de opstillede krav til funktionalitet, betalingshåndtering, opskænkningstid og doseringspræcision.

På baggrund af projektets resultater vurderes det derfor, at den udviklede løsning opfylder projektets formål om at realisere et fungerende automatiseret drikkevaresystem.

Fremtidigt arbejde

Selvom den udviklede prototype opfylder de opstillede krav, er der flere muligheder for videreudvikling. En naturlig udvidelse er at forbedre systemets skalerbarhed ved at understøtte flere doseringsenheder og flere samtidige brugere. Dette vil kræve yderligere udvikling af backendens håndtering af ordrer samt en mere avanceret kommunikationsarkitektur mellem delsystemerne.

På hardware-siden kan doseringssystemet forbedres gennem mere præcise ventiler og sensorer, hvilket kan reducere variationen i doseringsresultaterne. Desuden kan skænkningsalgoritmen optimeres for at håndtere forskellige drikketyper med forskellige fysiske egenskaber.

Endelig kan brugerinteraktionen videreudvikles med funktioner såsom brugerprofiler, historik over tidligere køb og mulighed for fjernadministration af drikkekataloget.

Den nuværende arkitektur vurderes dog at danne et godt fundament for disse udvidelser, da ansvarsfordelingen mellem systemets delkomponenter allerede er tydeligt defineret.

Bibliografi

[1] Rick Danaik og Joseph Feller. *Artificial Intelligence at Ringling: Framework for AI Fluency*. Accessed: 2026-07-01. URL: https://ringling.libguides.com/ai/framework.

[2] Steve Klabnik m.fl. *The Rust Programming Language*. Accessed: 2025-12-29. 2025. URL: https://doc.rust-lang.org/stable/book/.

Kapitel 10

Appendiks

10.1 Krav til rapport og supplerende materiale

En aflevering i forbindelse med et projekt består af en rapport (PDF) samt en række bilag, typisk pakket sammen in en zip-fil. Rapport og bilag afleveres som angivet i forbindelse med projektet på f.eks. Wiseflow eller Brightspace, hvor rapporten afleveres som den primære *Hovedopgave*, mens bilag afleveres som det sekundære *Supplerende materiale*.

10.1.1 Antal tegn

Omfanget af projektrapporten må ikke overstige 30 normalsider tekst. En normalside består af 2400 tegn med mellemrum, dvs. projektrapporten må fylde 72.000 tegn. Forside, resumé/abstract, indholdsfortegnelse, referenceliste, bilagsfortegnelse, tegn i figurer og tabeller samt bilag tæller ikke med i de 72.000 tegn.

10.1.2 Supplerende materiale

Rapporten kan suppleres med bilag med tekniske eller processmæssige detaljer, som der ikke er plads til i hovedrapporten. Bilagene henvender sig til en teknisk kynding på samme faglige niveau som forfatter og med disse samt rapporten, bør vedkommende være i stand til at kunne videreudvikle og/eller vedligeholde projektets produkt.

Eksempel på tekniske bilag:

- **Kravspecifikation** med alle funktionelle, ikke funktionelle krav, samt prioritering.
- **Risikomatrice** som opdateres i løbet af projektet
- **Analyserapporter** som beskriver analyser og eksperimenter anvendt i forbindelse med arkitektur og design.
- **Testdokumenter**, herunder integrationstests, udført accepttest, usability studier etc.
- **Implementeringsspecifikke filer** herunder kildekode, diagrammer, printudlæg, beregninger etc.
- **Datakilder**, herunder træningsdata, testdata mm.

Eksempel på procesrelaterede bilag:

- **Procesbeskrivelse**
- **Samarbejdsaftale**
- **Mødereferater**
- **Tidsplaner**, herunder den oprindelige og den endelige

- **Logbøger**

10.2 Sidebudget for rapport

Kapitel	**Sider**
Introduktion	2-4
Udviklingsproces og metode	1-2
Brugerkrav	4-6
Systemarkitektur	3-4
Design og Implementering	6-10
Systemintegrationstest	1-2
Brugeraccepttest	1-4
Diskussion	1-2
Konklusion	1-2
Total	30

Tabel 10.1: Sidebudget for rapport

10.3 Relevante rapportsektioner og diagrammer

10.3.1 Software Engineering-projekt

Kapitel	Sektion	Relevante figurer
Introduction	Background	
Introduction	Existing Work	
Introduction	Conceptual Overview of System	Rich Picture
Introduction	Problem Statement	
Methodology and Development Process	Project Model	
Methodology and Development Process	Development Methods	
User Requirements	Actors and Use Case Context	UML Actor-context diag.
User Requirements	Requirements Elicitation and Analysis	
User Requirements	Requirements Specification	UML Use Case diag.
User Requirements	Requirements Prioritization	MoSCoW Matix
User Requirements	Domain Model of the System	UML Domain Model
System Architecture	System Context and Scope	C4 Context Diag.
System Architecture	Technology and Risk Assessment	Risk Matrix
System Architecture	System Structure and Interfaces	C4 Container Diag., sys-level UML SD, ER diag.
Subsystem Design and Implementation	Subsystem X - Design and Implementation	C4 Comp diag of subsys., UML CD, SD, STM, code
Subsystem Design and Implementation	Subsystem X - Test and Results	Unit and local integr. test results and metrics
System Integration Testing	Integration Tests for Use Case Y	
User Acceptance Testing	Test Setup	
User Acceptance Testing	Results and Discussion	
Evaluation and Discussion	Discussion of Test Results	
Evaluation and Discussion	Discussion of System Architecture and Design	
Conclusion	Conclusion	
Conclusion	Future Work	

Tabel 10.2: Relevant sections and diagrams for Software Engineering Projects

10.3.2 Embedded Software Engineering-projekt

Kapitel	Sektion	Relevante figurer
Introduction	Background	
Introduction	Existing Work	
Introduction	Conceptual Overview of System	Rich Picture
Introduction	Problem Statement	
Methodology and Development Process	Project Model	
Methodology and Development Process	Development Methods	
User Requirements	Actors and Use Case Context	UML Actor-context diag.
User Requirements	Requirements Elicitation and Analysis	
User Requirements	Requirements Specification	UML Use Case diag.
User Requirements	Requirements Prioritization	MoSCoW Matix
User Requirements	Domain Model of the System	UML Domain Model
System Architecture	System Context and Scope	SysML BDD
System Architecture	Technology and Risk Assessment	Risk Matrix
System Architecture	System Structure and Interfaces	UML Deployment diag., sys-level UML SD + HW I/F and protocol desc.
Subsystem Design and Implementation	Subsystem X - SW Design and Implementation	Subsys UML Comp diag., SysML IBD, UML CD, SD, STM, (ACT), code
Subsystem Design and Implementation	Subsystem X - HW Design and Implementation	Analysis, schematics, PCB Layout
Subsystem Design and Implementation	Subsystem X - SW Test and Results	Unit test results and metrics
Subsystem Design and Implementation	Subsystem X - HW Test and Results	Test design and results
Subsystem Design and Implementation	Subsystem X - HW/SW Test and Results	Local HW/SW integration tests and results
System Integration Testing	Integration Tests for Use Case Y	
User Acceptance Testing	Test Setup	
User Acceptance Testing	Results and Discussion	
Evaluation and Discussion	Discussion of Test Results	
Evaluation and Discussion	Discussion of System Architecture and Design	
Conclusion	Conclusion	
Conclusion	Future Work	

Tabel 10.3: Relevant sections and diagrams for Embedded Software Engineering Projects

10.3.3 Electronics Engineering-projekt

Kapitel	Sektion	Relevante figurer
Introduction	Background	
Introduction	Conceptual Overview of System	Rich Picture
Introduction	Problem Statement	
Methodology and Development Process	Project Model	
Methodology and Development Process	Development Methods	
User Requirements	Actors and Use Case Context	UML Actor-context diag.
User Requirements	Requirements Elicitation and Analysis	
User Requirements	Requirements Specification	UML Use Case diag.
User Requirements	Requirements Prioritization	MoSCoW Matix
System Architecture	System Context and Scope	SysML BDD
System Architecture	Technology and Risk Assessment	Risk Matrix
System Architecture	System Structure and Interfaces	Sys-level SysML IBD,UML SD + HW I/F and protocol desc.
Subsystem Design and Implementation	Subsystem X - SW Design and Implementation	Subsys SysML IBD, UML ACT, STM, (CD), (SD), code
Subsystem Design and Implementation	Subsystem X - HW Design and Implementation	Analysis, schematics, PCB Layout
Subsystem Design and Implementation	Subsystem X - SW Test and Results	Unit test results and metrics
Subsystem Design and Implementation	Subsystem X - HW Test and Results	Test design and results
Subsystem Design and Implementation	Subsystem X - HW/SW Test and Results	Local HW/SW integration tests and results
System Integration Testing	Integration Tests for Use Case Y	
User Acceptance Testing	Test Setup	
User Acceptance Testing	Results and Discussion	
Evaluation and Discussion	Discussion of Test Results	
Evaluation and Discussion	Discussion of System Architecture and Design	
Conclusion	Conclusion	
Conclusion	Future Work	

Tabel 10.4: Relevant sections and diagrams for Electronics Engineering Projects

10.4 Code Listing Examples

```
int main() {
    std::vector<int> v = {1,2,3};
    int* p = &v[0]; // Set ptr
    v.push_back(4); // May reallocate
    std::cout << *p << "\n"; // UB
}
```

Kodeudsnit 10.1: Unboxed code listing

Reference to code listing: Kodeudsnit 10.1

Boxed Code Listing Header

```
int main() {
    std::vector<int> v = {1,2,3};
    int* p = &v[0]; // Set ptr
    v.push_back(4); // May reallocate
    std::cout << *p << "\n"; // UB
}
```

Vector push-back in C++

Kodeudsnit 10.2: Boxed Code Listing

Reference to code listing: Kodeudsnit 10.2

Bash Listing Example Header

```
au276283@d43582 classicthesis % git checkout -b main
Switched to a new branch 'main'
au276283@d43582 classicthesis % git push --set-upstream originAU main
Enumerating objects: 848, done.
Counting objects: 100% (848/848), done.
Delta compression using up to 8 threads
```

Below comment

Kodeudsnit 10.3: Bash Listing Example

This is a citation: [2]

Boxed Side-by-Side Code Listing Header

```
int main() {
    std::vector<int> v = {1,2,3};
    int* p = &v[0];
    v.push_back(4); // May reallocate
    std::cout << *p << "\n"; // UB
}
```

Left Title

```
fn main() {
    let mut v = vec![1, 2, 3];
    let p = &v[0];
    v.push(4);
    println!("{p}");
}
```

Right Title

Kodeudsnit 10.4: Boxed Side-by-Side Code Listing

Boxed Two-Row Code Listing Header

```cpp
int main() {
    std::vector<int> v = {1,2,3};
    int* p = &v[0];
    v.push_back(4); // May reallocate
    std::cout << *p << "\n"; // UB
}
```

Upper Section

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    let p = &v[0];
    v.push(4);
    println!("{p}");
}
```

Lower Section

Kodeudsnit 10.5: Boxed Two Row Code Listing

10.5 Table examples

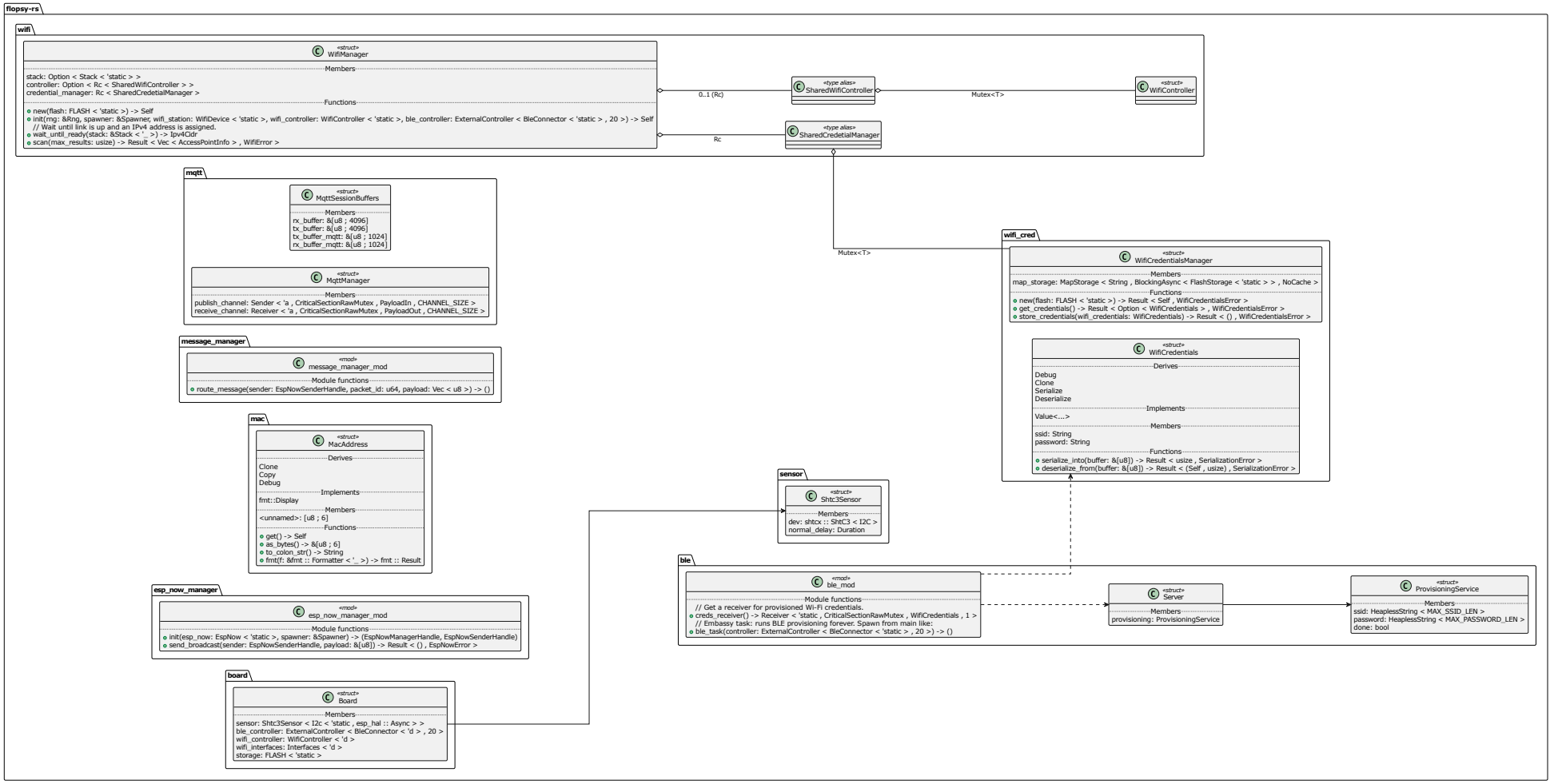
feature	no_std	std
heap (dynamic memory)	*	✓
collections (Vec, BTreeMap, etc)	**	✓
stack overflow protection	×	✓
runs init code before main	×	✓
libstd available	×	✓
libcore available	✓	✓
writing firmware, kernel, or bootloader code	✓	×

* possible with an allocator (`alloc`) ** many collections require `alloc`

Tabel 10.5: Table example with footnote

See Tabel 10.5

10.6 Rotated Full-page Image Example



Figur 10.1: Figure Example full-page landscape